
Rapport de stage

Félix Belanger

felix.belanger@universite-paris-saclay.fr

16 Avril 2026

Table des matières

1	Introduction	2
2	Présentation des types ensemblistes	2
2.1	Définition d'un type ensembliste	2
2.2	Existence d'une forme normale disjonctive	3
3	Génération d'un témoin	4
3.1	Définition d'un témoin	4
3.2	Définition de $\text{type_of}(w)$ et $w \gg t$	5
4	Algorithme	5
4.1	Présentation de l'algorithme	5
4.2	Preuve de terminaison de l'algorithme	6
4.3	Preuve de sûreté	7
5	Extensions	7
5.1	Tags	7
5.2	Records	7
6	Implémentation	8
6.1	Cas de bases	8
6.1.1	Int	8
6.1.2	Enum	8
6.1.3	Arrow	8
6.2	Constructeurs	9
6.2.1	N-uplets	9
6.2.2	Tags	9
6.2.3	Records	9
7	Variables de type	10
7.1	Définition	10
7.2	Présentation de l'algorithme	11
7.2.1	Création du tableau de contraintes	12
7.2.2	Créer ν respectant ces contraintes	13
7.3	Preuve de terminaison	16
7.4	Preuves de sûretés	16
8	Un cas concret	17
9	Conclusion et perspectives	18
	Bibliographie	19

1 Introduction

Le but du stage a été de créer une fonction permettant, pour tout type non-vide, de trouver un habitant de ce type, appelé témoin. Ce témoin permet d'afficher un exemple lors de la diffusion d'un message d'erreur pour mauvais typage.

Exemple

On a une fonction $f : \text{Int} \rightarrow \text{Int}$ et une variable $x : \text{Int} \vee \text{Bool}$. Alors l'application $f(x)$ est mal typée, car $\text{Int} \vee \text{Bool}$ n'est pas un sous-type de Int . On cherche donc un témoin de $(\text{Int} \vee \text{Bool}) \setminus \text{Int} = \text{Bool}$, ce qui peut rendre True ou False.

Pour des types plus complexes, on veut automatiser le processus, ce qui amène à l'algorithme « Witness » expliqué ici.

2 Présentation des types ensemblistes

2.1 Définition d'un type ensembliste

Les types utilisés dans ce rapport ont été introduits par M. Laurent, K. Nguyen et G. Castagna dans « A Gentle Introduction to Semantic Subtyping » (Castagna 2005) et « Implementing Set-Theoretic Types » (Laurent et Nguyen 2025) comme des types ensemblistes. On les définit comme :

$$t ::= b \mid \alpha \mid t \rightarrow t \mid t \times t \mid t \vee t \mid t \wedge t \mid \neg t \mid \emptyset \mid \mathbb{1} \quad (1)$$

Où $b \in \mathcal{B}$ sont les cas de bases (en particuliers les constantes $c \in \mathcal{C}$), $\alpha \in \mathcal{V}$ les variables de types, $\emptyset$ le type vide et $\mathbb{1}$ le supertype de tout les types.

Exemple

$t = (\text{Int}, (\text{True} \rightarrow \text{False})) \vee 42$ est un type correct dans notre syntaxe.

On définit $\llbracket t \rrbracket$ comme l'ensemble des valeurs contenues dans t , ce qui nous permet de définir aussi la relation de sous-typage $t_1 \leq t_2$ qui est vrai si et seulement si $\llbracket t_1 \rrbracket \subseteq \llbracket t_2 \rrbracket$. On peut noter qu'il existe des cas, comme Bool et Int , où $\text{Bool} \not\leq \text{Int}$ et $\text{Int} \not\leq \text{Bool}$. On a aussi que $\forall t, \emptyset \leq t \leq \mathbb{1}$. On peut enfin définir l'égalité sémantique $t_1 \simeq t_2$ comme étant vrai si et seulement si $t_1 \leq t_2$ et $t_2 \leq t_1$.

Comme ces types sont définis co-inductivement, ils peuvent être définis comme des arbres infinis réguliers et contractifs.

Définition 2.1

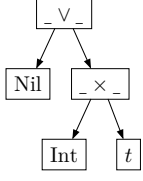
Un arbre est dit **régulier** si il a un nom fini de sous-termes possibles.

Définition 2.2

Un arbre est dit **contractif** si une branche infinie passe forcément par un constructeur (ici, $\rightarrow$ et $\times$).

Exemple

$t = \text{Nil} \vee (\text{Int} \times t)$ sous forme d'arbre:



On peut aussi représenter cela sous la forme d'un graphe cyclique en remplaçant la feuille t par une flèche vers $_ \vee _$. Ceci nous permet d'avoir une définition de type qui n'a pas besoin de définitions explicites pour la récursion.

Enfin, on considère les types infinis comme vides, car ils sont inconstructibles en pratique (ex : $t = (\text{Int}, t) \leq 0$).

Dans un premier temps, nous allons ignorer les variables de type, nous y reviendront au chapitre 7.

Définition 2.3

D'après la Définition C.13 de « Polymorphic Functions with set-theoretic types » (Castagna et al. 2013), un socle $\mathfrak{J} \subset \mathcal{T}$ est un ensemble de types avec les propriétés suivantes :

- $\mathfrak{J}$ est fini,
- $\mathfrak{J}$ est clos sous les connecteurs booléens ($\vee$, $\wedge$ et $\neg$).
- Si $t_1 \times t_2 \in \mathfrak{J}$ ou $t_1 \rightarrow t_2 \in \mathfrak{J}$ alors $t_1 \in \mathfrak{J}$ et $t_2 \in \mathfrak{J}$.

De plus, si on a $S : \{s \mid s \sqsubseteq t\}$ l'ensemble des sous-arbres de t , alors $\exists \mathfrak{J}$ tel que $S \subseteq \mathfrak{J}$.

Cette définition nous sera utile plus tard, lors de la création de l'algorithme de génération du témoin.

2.2 Existence d'une forme normale disjonctive

En pratique, un type est toujours encodé sous la forme d'une DNF (forme normale disjonctive) :

$$t ::= \bigvee_i b_i \vee \bigvee_i \left(\bigwedge_j (p_1^{ij} \rightarrow p_2^{ij}) \wedge \bigwedge_j \neg(n_1^{ij} \rightarrow n_2^{ij}) \right) \vee \bigvee_i \left(\bigwedge_j (p_1^{ij} \times p_2^{ij}) \wedge \bigwedge_j \neg(n_1^{ij} \times n_2^{ij}) \right) \quad (2)$$

Avec des atomes positifs, nommés p , et des atomes négatifs, nommés n . On notera que, dû à la récursivité de notre définition, on peut avoir des opérateurs d'union sous les opérateurs d'intersection (si on développe les types flèches et tuples). On appellera quand même cette forme une DNF par la suite, par mesure de simplicité. Les atomes de bases ne sont représentés que comme des unions car il sont très simplement simplifiables.

On peut simplifier cette définition en transformant la partie sur les tuples en une union d'unions :

Lemme 2.4

$$\forall t, t \leq 1 \times 1 \Rightarrow \exists (t_1^i, t_2^i), t \simeq \bigvee_i (t_1^i, t_2^i)$$

Démonstration: Si $t \leq 1 \times 1$ alors :

$$\begin{aligned}
t &= \bigvee \left(\bigwedge_i (p_1^i, p_2^i) \wedge \bigwedge_j \neg(n_1^j, n_2^j) \right) \\
&= \bigvee \left(\left(\bigwedge_i (p_1^i, p_2^i) \right) \setminus \bigvee_j (n_1^j, n_2^j) \right) \\
&= \bigvee \left(\left(\bigwedge_i p_1^i, \bigwedge_i p_2^i \right) \setminus \bigvee_j (n_1^j, n_2^j) \right) \text{ par commutativité de } \wedge
\end{aligned}$$

Montrons par récurrence sur $\bigvee_j (n_1^j, n_2^j)$ que t est bien une union :

On a bien $(\bigwedge_i p_1^i, \bigwedge_i p_2^i)$ qui est une union d'un seul élément.

On a maintenant $\bigvee_i (t_1^i, t_2^i)$ une union, montrons que $\bigvee_i (t_1^i, t_2^i) \setminus (n_1, n_2)$ est une union :

$$\begin{aligned}
\bigvee_i (t_1^i, t_2^i) \setminus (n_1, n_2) &= \bigvee_i ((t_1^i, t_2^i) \setminus (n_1, n_2)) \\
&= \bigvee_i ((t_1^i \setminus n_1, t_2^i \setminus n_2) \vee (t_1^i, t_2^i \setminus n_2)) \text{ qui est une union.}
\end{aligned}$$

Donc par récurrence, $\forall t, t \leq 1 \times 1 \Rightarrow \exists (t_1^i, t_2^i), t \simeq \bigvee_i (t_1^i, t_2^i)$. $\square$

Ainsi, on peut réécrire la définition vu plus haut :

$$t ::= \bigvee_i b_i \vee \bigvee_i \left(\bigwedge_j (p_1^{ij} \rightarrow p_2^{ij}) \wedge \bigwedge_j \neg(n_1^{ij} \rightarrow n_2^{ij}) \right) \vee \bigvee_i (t_1^i \times t_2^i) \quad (3)$$

3 Génération d'un témoin

3.1 Définition d'un témoin

Un témoin est un habitant de notre type, c'est à dire une valeur constante appartenant à $\llbracket t \rrbracket$. Notre but est, pour chaque cas de base, de trouver un habitant de ce cas, et pour chaque constructeur, de chercher récursivement un témoin pour chacune de ses parties.

Exemple

16 est un témoin de $\text{Int} \vee \text{Bool}$.

Pour trouver un habitant d'une fonction f de type $t_1 \rightarrow t_2$, il faut un langage qui puisse prendre un habitant de t_1 et renvoyer le résultat de $f(\text{Witness}(t_1)) = \text{Witness}(t_2)$. On pourrait utiliser des λ termes mais cela serait complexes pour les types rékursifs. Enfin, savoir que $42 \rightarrow 43$ est un habitant de $\text{Any} \rightarrow \text{Int}$ ne nous aide pas vraiment. Le procédé est donc complexe est contre-productif, nous avons donc choisis de considérer les types flèches (de la forme $\bigvee_i \left(\bigwedge_j p_1^{ij} \rightarrow p_2^{ij} \wedge \bigwedge_j \neg(n_1^{ij} \rightarrow n_2^{ij}) \right)$) comme des cas de base. On notera

$$A ::= \bigwedge_i (p_1^i \rightarrow p_2^i) \wedge \bigwedge_j \neg(n_1^j \rightarrow n_2^j) \quad (4)$$

Cela va nous permettre de réécrire une nouvelle fois notre définition de type :

$$t ::= \bigvee_i b_i \vee \bigvee_i A_i \vee \bigvee_i (t_1^i \times t_2^i) \quad (5)$$

Un témoin est donc soit une constante d'un cas de base, soit un type flèche, soit un tuple. On propose donc comme type témoin :

$$w ::= c \mid A \mid (w, w) \quad (6)$$

Exemple

$w = (42, (\text{True}, \text{False}))$ est un témoin correct dans notre syntaxe.

3.2 Définition de $\text{type_of}(w)$ et $w \gg t$

On peut donc créer l'algorithme $\text{type_of}(w)$ qui pour tout témoin w renvoie le type t contenant uniquement w :

$$\frac{c = b}{\vdash \text{type_of}(c) = b} \text{Base}_{\text{type_of}}$$

$$\frac{\frac{w = A}{\vdash \text{type_of}(w) = A} \text{type_of} \quad \frac{\vdash \text{type_of}(w_1) = t_1 \quad \vdash \text{type_of}(w_2) = t_2}{\vdash \text{type_of}(w_1, w_2) = (t_1 \times t_2)} \times_{\text{type_of}}}{\vdash \text{type_of}(w) = A} \text{type_of}$$

On peut étendre cet algorithme afin que pour tout témoin w , il renvoie le type t tel que $w \leq t$. On l'appellera $w \gg t$ et aura les règles suivantes :

$$\frac{c \in \llbracket b \rrbracket}{\vdash c \gg b} \text{Base}_w \quad \frac{w \leq A}{\vdash w \gg A} \xrightarrow{w}$$

$$\frac{\vdash w_1 \gg t_1 \quad \vdash w_2 \gg t_2}{\vdash (w_1, w_2) \gg (t_1 \times t_2)} \times_w \quad \frac{\vdash w \gg t' \quad t' \leq t}{\vdash w \gg t} \text{Sous-type}_w$$

Exemple

On veut montrer que $w = (3, (\text{True}, \text{Int} \rightarrow \text{Int}))$ est bien un témoin de $t = (\text{Int}, (\text{Bool}, 0 \rightarrow 1))$:

$$\frac{\frac{3 \in \llbracket \text{Int} \rrbracket}{\vdash 3 \gg \text{Int}} \text{Base}_w \quad \frac{\frac{\text{True} \in \llbracket \text{Bool} \rrbracket}{\vdash \text{True} \gg \text{Bool}} \text{Base}_w \quad \frac{\text{Int} \rightarrow \text{Int} \leq 0 \rightarrow 1}{\vdash \text{Int} \rightarrow \text{Int} \gg 0 \rightarrow 1} \xrightarrow{w}}{\vdash (3, (\text{True}, \text{Int} \rightarrow \text{Int})) \gg (\text{Int}, (\text{Bool}, 0 \rightarrow 1))} \times_w$$

4 Algorithme

4.1 Présentation de l'algorithme

On peut maintenant décrire le fonctionnement de l'algorithme qui pour chaque type t associe un témoin w (qu'on notera $t \rightsquigarrow w$ et appellera $\text{Witness}(t)$) qui a les règles suivantes :

$$\frac{w = c \leq b}{\Delta \vdash_s b \rightsquigarrow w} \text{Base}_t \quad \frac{w \leq A}{\Delta \vdash_s A \rightsquigarrow w} \xrightarrow{t} \quad \frac{\Delta \vdash_m t_1 \rightsquigarrow w_1 \quad \Delta \vdash_m t_2 \rightsquigarrow w_2}{\Delta \vdash_s t_1 \times t_2 \rightsquigarrow (w_1, w_2)} \times_t$$

$$\frac{\Delta \vdash_s t_1 \rightsquigarrow w}{\Delta \vdash_s t_1 \vee t_2 \rightsquigarrow w} \vee_{1_t} \quad \frac{\Delta \vdash_s t_2 \rightsquigarrow w}{\Delta \vdash_s t_1 \vee t_2 \rightsquigarrow w} \vee_{2_t} \quad \frac{\Delta \cup \{t\} \vdash_s t \rightsquigarrow w}{\Delta \vdash_m t \rightsquigarrow w} t \notin \Delta$$

Ici, Δ est l'ensemble des types déjà visité par l'algorithme. Ainsi, si l'on retombe sur le même type, on sait que celui-ci est infini, donc vide (comme expliqué ici **RAJOUTER LIEN**). On interdit donc de repasser plusieurs par un type déjà visité avec la règle $t \notin \Delta$.

Exemple

: On a $t = \text{Int} \times ((\text{Int} \rightarrow \text{Bool}) \vee \text{Nil})$

Cela nous donne l'arbre :

$$\frac{\frac{w_1 = 42 \leq \text{Int}}{\{\text{Int}\} \vdash_s \text{Int} \rightsquigarrow w_1} \text{Base}_t \quad \frac{\frac{w_2 = \text{Int} \rightarrow 0 \leq \text{Int} \rightarrow \text{Bool}}{\Delta \vdash_s \text{Int} \rightarrow \text{Bool} \rightsquigarrow w_2 = \text{Int} \rightarrow 0} \rightarrow_t \quad \frac{\Delta = \{(\text{Int} \rightarrow \text{Bool}) \vee \text{Nil}\} \vdash_s (\text{Int} \rightarrow \text{Bool}) \vee \text{Nil} \rightsquigarrow w_2}{t \notin \Delta} \vee_{1_t}}{\frac{\vdash_m \text{Int} \rightsquigarrow w_1 \quad \vdash_m t = (\text{Int} \rightarrow \text{Bool}) \vee \text{Nil} \rightsquigarrow w_2}{\vdash_s \text{Int} \times ((\text{Int} \rightarrow \text{Bool}) \vee \text{Nil}) \rightsquigarrow (w_1, w_2)} \times_t}$$

On peut ainsi vérifier que $w = (42, \text{Int} \rightarrow 0)$ est bien un témoin de $t = \text{Int} \times ((\text{Int} \rightarrow \text{Bool}) \vee \text{Nil})$:

$$\frac{\frac{42 \in \llbracket \text{Int} \rrbracket}{\vdash 42 \gg \text{Int}} \text{Base}_w \quad \frac{\frac{\text{Int} \rightarrow 0 \leq (\text{Int} \rightarrow \text{Bool})}{\vdash \text{Int} \rightarrow 0 \gg (\text{Int} \rightarrow \text{Bool})} \rightarrow_w \quad \frac{(\text{Int} \rightarrow \text{Bool}) \leq (\text{Int} \rightarrow \text{Bool}) \vee \text{Nil}}{\vdash \text{Int} \rightarrow 0 \gg (\text{Int} \rightarrow \text{Bool}) \vee \text{Nil}} \text{Sous-type}_w}{\vdash (42, \text{Int} \rightarrow 0) \gg \text{Int} \times ((\text{Int} \rightarrow \text{Bool}) \vee \text{Nil})} \times_w$$

4.2 Preuve de terminaison de l'algorithme

Théorème 4.1

Pour tout type t non-vide, $t \rightsquigarrow w$ termine.

Démonstration: On a, par définition, que $\Delta \subseteq \mathfrak{J}$, donc Δ est de taille finie. On pose u le nombre d'unions directes dans notre terme, avec $u = 1$ si l'on choisit le membre de gauche, et $u = u - 1$ si l'on choisit le membre de droite, et $s ::= \begin{cases} 1 & \text{si } \vdash_s \\ 0 & \text{si } \vdash_m \end{cases}$.

Alors on peut construire le nombre $(|\mathfrak{J} \setminus \Delta|, u, s)$. Montrons que, quelque que soit la règle suivie, ce nombre décroît sur l'ordre lexicographique, assurant donc la terminaison de l'algorithme :

Pour Base_t : il suffit de prendre un atome de $\bigvee_i b_i$, ce qui se fait en temps fini.

Pour $\rightarrow_t$: il suffit de prendre un atome de A , ce qui se fait aussi en temps fini.

Pour $\times_t$: s passe de 1 à 0 car on passe d'une règle $\vdash_s$ à des règles $\vdash_m$, on a donc bien $(|\mathfrak{J} \setminus \Delta|, u, 0) <_{\text{lex}} (|\mathfrak{J} \setminus \Delta|, u, 1)$.

Pour $\vee_{1_t}$: u passe à 1, donc on a bien $(|\mathfrak{J} \setminus \Delta|, 1, 1) <_{\text{lex}} (|\mathfrak{J} \setminus \Delta|, u, 1)$.

Pour $\vee_{2_t}$: u passe à $u - 1$, donc on a bien $(|\mathfrak{J} \setminus \Delta|, u - 1, 1) <_{\text{lex}} (|\mathfrak{J} \setminus \Delta|, u, 1)$.

Pour $t \notin \Delta$: u est susceptible d'augmenter, mais Δ augmente, donc $|\mathfrak{J} \setminus \Delta|$ décroît de 1, on a donc bien $(|\mathfrak{J} \setminus \Delta| - 1, u', 1) <_{\text{lex}} (|\mathfrak{J} \setminus \Delta|, u, 0)$.

Donc par induction fondée sur l'ordre lexicographique de $(|\mathfrak{J} \setminus \Delta|, u, s)$, pour tout type $t \not\leq 0$, $t \rightsquigarrow w$ termine. $\square$

4.3 Preuve de sûreté

Théorème 4.2

Pour tout type t non-vide, si $\emptyset \vdash_s t \rightsquigarrow w$ alors $w \gg t$. En d'autres termes, $\forall t \not\leq 0, \text{type_of}(\text{Witness}(t)) \leq t$.

Démonstration: On veut prouver $P(t) ::= \text{Si } \emptyset \vdash_s t \rightsquigarrow w \text{ alors } w \gg t$:

On prouve par induction sur les règles d'inférences de $t \rightsquigarrow w$:

Pour Base_t : Supposons $\vdash_s b \rightsquigarrow w$, alors par Base_t , $w = c \leq b$, donc $w = c \in \llbracket b \rrbracket$, donc $w \gg b$.

Pour $\rightarrow_t$: Supposons $\vdash_s A \rightsquigarrow w$, alors par $\rightarrow_t$, $w = w_1 \rightarrow w_2 \leq A$, donc par $\rightarrow_w$, $w \gg A$.

Supposons maintenant $P(t_1)$ et $P(t_2)$.

Pour $\times_t$: Supposons $(t_1 \times t_2) \rightsquigarrow (w_1, w_2)$, alors par $\times_t$, $t_1 \rightsquigarrow w_1$ et $t_2 \rightsquigarrow w_2$, donc par $P(t_1)$ et $P(t_2)$, on a $w_1 \gg t_1$ et $w_2 \gg t_2$, donc par $\times_w$, $(w_1, w_2) \gg (t_1 \times t_2)$.

Pour $\vee_{1_t}$: Supposons $t_1 \vee t_2 \rightsquigarrow w$, alors par $\vee_{1_t}$, $t_1 \rightsquigarrow w$, donc par $P(t_1)$ on a $w \gg t_1$, or $t_1 \leq t_1 \vee t_2$ donc par Sous-type_w , $w \gg t_1 \vee t_2$.

Pour $\vee_{2_t}$: Supposons $t_1 \vee t_2 \rightsquigarrow w$, alors par $\vee_{2_t}$, $t_2 \rightsquigarrow w$, donc par $P(t_2)$ on a $w \gg t_2$, or $t_1 \leq t_1 \vee t_2$ donc par Sous-type_w , $w \gg t_1 \vee t_2$.

Donc pour tout type t non-vide, si $\emptyset \vdash_m t \rightsquigarrow w$ alors $w \gg t$. $\square$

5 Extensions

Des extensions ont été rajoutés aux types ensemblistes déjà présents afin de couvrir certaines types précis : les n-uplets, les tags et les records. Ceux-ci ne sont que des versions spécifiques des tuples, ils ne seront donc pas traités sur le plan théorique. De plus, nos cas de bases peuvent être de 2 types : Int et Enum. On peut donc étendre notre définition de témoin :

$$w ::= i \in \llbracket \text{Int} \rrbracket \mid e \in \llbracket \text{Enum} \rrbracket \mid A \mid (w, w, \dots, w) \mid \text{tag}(w) \mid \{l_i : w \dots l_n : w\} \quad (7)$$

5.1 Tags

Les atomes de tags sont des tuples avec un type Tag (équivalent à un type string) en premier membre et un type t en deuxième. Ils servent à marquer certains types pour ne pas être confondus (ex : $\text{foo}(\text{Int})$).

5.2 Records

Les atomes de records sont de la forme

$$r ::= \{\text{bindings} : (\text{Label} \times f) \text{ map}; \text{tail} : f; \text{exists} : (\text{Label Set} \times f) \text{ Map}\} \quad (8)$$

- Ici, le type f est une extension de notre type t qui inclus les types optionnels (ex : $f = \text{Int}?$). On le définit comme $f \in \mathbb{1} \cup \perp$, $\perp$ dénotant le type absent.
- Le bindings est un champ où l'on place toutes les variables nommées de notre record.

Exemple

$\{x : \text{Int}; y : \text{Int?}; \text{prédicat} : \text{Bool}\}$

- La tail f' indique si notre record est ouvert ou fermé (donc s'il peut avoir des variables supplémentaires non nommées).

Exemple

$\{x : \text{Int}; y : \text{Int};; \text{Int}?\}$ pour un système de coordonnées à au moins 2 dimensions.

- Pour chaque paire $(L_i : e_i)$ de exists, L_i indique les noms que la variables ne peut PAS prendre et $e_i \wedge f'$ indique son type.

Exemple

$\{x : \text{Int}; y : \text{Int};; \text{Any}?\;; \{w; z\} : \text{Bool}?\}$ pour indiquer que les variables w et z ne peuvent pas avoir le type Bool si elles existent.

6 Implémentation

L'algorithme va essayer de trouver un témoin dans chacune des 6 grandes catégories (Int, Enum, Arrows, Tag, Tuple, Record) dans cet ordre, et s'arrêter dès qu'un est trouvé. Si aucun témoin n'est trouvé, on considère le type comme vide et on lève une exception.

6.1 Cas de bases

6.1.1 Int

Les entiers sont représentés comme des intervalles. Il existe 3 cas possibles :

- $t =] - \infty; +\infty[$: l'algorithme retourne 42.
- $t =] - \infty; i]$ ou $t = [i; +\infty[$: retourne i .
- $t = [i_1; i_2]$: retourne i_1 .

6.1.2 Enum

Enum contient tout les types énumérés, incluant notamment les booléens et les chaînes de caractères. Il peuvent être définis de 2 manières :

- comme une union des constantes appartenant à t : on renvoie la première
- comme une union des constantes appartenant à $\neg t$: On retourne une constante de Enum de taille n , n n'étant la taille d'aucune des constantes de $\neg t$

Exemples

1. $\text{True} \mid \text{False}$ donnera True.
2. $\text{Enum} \setminus (a \vee ba \vee \text{toto})$ donnera abc.

6.1.3 Arrow

Comme dit plus haut, on considère les atomes de la forme $\bigwedge_i (t_1^i \rightarrow t_2^i) \wedge \bigwedge_j \neg(t_1 \rightarrow t_2)$ comme des cas de base. On va donc prendre tout les atomes positifs de t , et y rajouter un par un les atomes négatifs de t . Si au moment d'un ajout, l'intersection des atomes positifs et négatifs donne un sous-type de t , c'est notre témoin.

6.2 Constructeurs

6.2.1 N-uplets

Les n-uplets peuvent, comme les Enum, être défini comme les éléments présents dans t ou comme les éléments présents dans $\neg t$.

- Si on a la liste des éléments présents dans t , on choisit l'élément e de plus petite arité dont aucun élément n'est vide, et on renvoie un n-uplet de la même taille et dont chaque élément est le témoin du type à la même position dans e .
- Si on a la liste des éléments présents dans $\neg t$, on trouve le plus petit entier n tel qu'il n'y ai aucun n-uplet de cette arité dans la liste, et on en renvoie un témoin (usuellement le n-uplet $(0,1,\dots,n)$)

Exemples

1. $(\text{Int}, \text{Bool})$ donnera $(42, \text{True})$
2. $\text{Tuple} \setminus ((\text{Int},) \vee (\text{Int}, \text{Bool}))$ donnera $(0, 1, 2)$

6.2.2 Tags

Les Tags sont traités de la même manière que les n-uplets de taille 2, à l'exception que l'on n'opère pas de récursion sur le premier membre.

Exemples

1. $\text{foo}(\text{Int})$ donnera $\text{foo}(42)$.
2. $\text{Tag} \setminus \text{foo}(\text{Int})$ donnera $\text{a}(16)$.

6.2.3 Records

Pour générer un témoin, on va trouver un atome non vide de notre record, et opérer comme suit :

- Pour chaque paire $(l_i : f_i)$ de bindings, si f_i est requis, alors on cherche récursivement un témoin w_i de f_i et ajouter $(l_i : w_i)$ au bindings de notre témoin w .

Exemple

$\{x : \text{Int}; y : \text{Int?}; \text{prédicat} : \text{Bool}\}$ donnera $\{x : 42; \text{prédicat} : \text{True}\}$.

- Si la tail f' est requise, on l'ajoute dans la tail de w .

Exemples

1. $\{x : \text{Int};; \text{Bool}\}$ donnera $\{x : 42;; \text{True}\}$.
2. $\{x : \text{Int};; \text{Bool?}\}$ donnera $\{x : 42\}$.

- Pour chaque paire $(L_i : e_i)$ de exists, si $e_i \wedge f$ est requis, alors on crée un label $l'_i \notin L_i$, on cherche récursivement un témoin w'_i de $e_i \wedge f$ et on ajoute $(l'_i : w'_i)$ **au bindings**.

Exemple

$\{x : \text{Int}; y : \text{Int} ;; \text{Any?} ;; \{w; z\} : \text{Bool}\}$ donnera $\{x : 42; y : 42; a : \text{True}\}$

7 Variables de type

7.1 Définition

On veut maintenant intégrer les variables de type (aussi appelés types polymorphes) dans notre recherche de témoin. On commence donc par étendre notre définition de t :

$$t ::= b \mid \alpha \mid t \rightarrow t \mid t \times t \mid t \vee t \mid t \wedge t \mid \neg t \mid 0 \mid 1 \quad (9)$$

Définition 7.1

Un type t est **non-vide** si et seulement si il existe une substitution σ tel que $t\sigma \not\leq 0$.

On veut donc renvoyer une substitution σ et un témoin de $t\sigma$, avec comme conditions que $\text{Vars}(t\sigma) = \emptyset$, $t\sigma \not\leq 0$ et $\text{Witness}(t\sigma) \gg t\sigma$.

Notre témoin ressemblera donc à :

$$w' ::= (\sigma, w) \quad (10)$$

Avec σ l'ensemble des substitutions et w le témoin présenté à l'Équation 6.

Définition 7.2

Le **tallying** (noté $\text{Tally}(s_1, t_1) = [[(\alpha^1, \sigma_1(\alpha^1)), \dots, (\alpha^k, \sigma_1(\alpha^k))], \dots, [(\alpha^1, \sigma_n(\alpha^1)), \dots, (\alpha^k, \sigma_n(\alpha^k))]]$) est un algorithme qui renvoie toutes les substitutions σ tel que $\forall \sigma_i \in \sigma, \forall j \leq n, s_j \sigma_i \leq t_j$. Toutes les substitutions sont de la forme $\{\alpha \mapsto (\alpha \vee t_{\text{inf}}) \wedge t_{\text{sup}}\}$ afin de borner efficacement l'algorithme.

Exemple

$$\begin{aligned} \text{Tally}((\alpha, \beta), 0) &= [[(\alpha, 0), (\beta, \beta)], \\ &\quad [(\alpha, \alpha), (\beta, 0)]] \\ &= [[\{\alpha \mapsto (0 \vee \alpha) \wedge 0\}, \{\beta \mapsto (0 \vee \beta) \wedge 1\}], \\ &\quad [\{\alpha \mapsto (0 \vee \alpha) \wedge 1\}, \{\beta \mapsto (0 \vee \beta) \wedge 0\}]] \end{aligned}$$

Note

Les variables de types ont un ordre total $\preceq$. Ainsi, une variable α^i ne peut pas avoir dans sa substitution les variables $\alpha^1, \dots, \alpha^{i-1}$, i.e $\forall i, j, \forall k < i, \alpha^k \notin \text{Vars}(\sigma_j(\alpha^i))$

Note

L'algorithme de Tallying renvoie une réponse complète, c'est-à-dire qu'il n'existe pas de substitutions qui ne soient pas des sous-cas d'une des solutions et qui soit une solution au problème de tallying.

Exemple

$$\begin{aligned} \text{Tally}(\alpha \setminus \beta, 0) &= [[(\alpha, \alpha \wedge \beta), (\beta, \beta)]] \\ &= [[\{\alpha \mapsto (0 \vee \alpha) \wedge \beta\}, \{\beta \mapsto (0 \vee \beta) \wedge 1\}]] \end{aligned}$$

Définition 7.3

On dit qu'une substitution σ_j est **débornée** pour une variable α^i si on a $\inf_j^i \leq 0$ et $\sup_j^i \geq 1$. Cela veut dire que cette variable ne joue pas de rôle dans la substitution.

En pratique on va représenter le tallying sous la forme d'un tableau :

tally	α^1	α^2	...	α^k
σ_1	i_1^1, s_1^1	i_1^2, s_1^2	...	i_1^k, s_1^k
σ_2	i_2^1, s_2^1	i_2^2, s_2^2	...	i_2^k, s_2^k
...	...	...	...	...
σ_n	i_n^1, s_n^1	i_n^2, s_n^2	...	i_n^k, s_n^k

avec i_j^i et s_j^i respectivement les bornes inférieures et supérieures de $\sigma_j(\alpha^i)$.

Lemme 7.4

Toutes les substitutions de $\text{Tally}(t, 0)$ avec t non vide **sont bornées sur au moins une variable**. Sinon, elle serait équivalente à la substitution identité $\{\}$, donc $t\{\} \leq 0$, donc t serait vide.

Exemple

Pour $\text{Tally}[(\alpha, \neg\alpha), 0]$:

	α	β
σ_1	$0, 0$	$0, 1$
σ_2	$0, 1$	$0, 0$

Pour $\text{Tally}[(\alpha \setminus \beta, 0)]$:

	α	β
σ_1	$0, \beta$	$0, 1$

7.2 Présentation de l'algorithme

On cherche à trouver la substitution σ tel que $\text{Vars}(t) = \emptyset$ et $t\sigma \not\leq 0$. On appelle cet algorithme $\text{polyw}(t)$.

On distingue 3 cas possibles :

- Si $\text{Vars}(t) = \emptyset$, on renvoie la substitution identité.
- Si $\text{Tally}(t, 0) = \emptyset$, le type n'a aucune substitution qui puisse le vider, on substitue donc toutes les variables par un type arbitraire sans variable (ici 0) : $\bigcup_{\alpha^i \in \text{Vars}(t)} \{\alpha^i \rightarrow 0\}$.
- Sinon, on crée la substitution $\sigma := \{\alpha^k \rightarrow \nu\}$ avec α^k et ν choisis correctement (et expliqués plus loin), et on renvoie $\sigma \cup \text{polyw}(t\sigma)$.

Pour choisir correctement α^k et ν dans le dernier cas, on va donc utiliser le tallying, afin de déterminer quelle substitution ne vide **pas** notre type t .

On commence par choisir α^k tel que celle-ci soit la plus grande variable de $\text{Vars}(t)$ selon l'ordre total $\preceq$. Notre substitution sera donc de la forme $\sigma := \{\alpha^k \rightarrow \nu\}$.

7.2.1 Création du tableau de contraintes

Une fois cela fait, on va créer un tableau contenant toutes les bornes dans lesquelles on ne veut **pas** que ν soit, et ceci en 2 étapes :

Colonne de α^k

On va d'abord prendre la colonne α^k dans le tableau du tallying :

	α^k
σ_1	i_1^k, s_1^k
σ_2	i_2^k, s_2^k
...	...
σ_n	i_n^k, s_n^k

Puis supprimer les substitutions débordées, car alors la variable α^k n'influe pas sur la substitution.

Exemple

Pour $(\alpha^1, \neg\alpha^1) \setminus (\alpha^2, \text{Int})$:

$\text{Tally}[(\alpha^1, \neg\alpha^1) \setminus (\alpha^2, \text{Int}), 0] =$

	α^1	α^2
σ_1	$0, 0$	$0, 1$
σ_2	$1, 1$	$0, 1$
σ_3	$\alpha^2 \wedge \text{Int}, \text{Int}$	$\neg \text{Int}, 1$

On ne garde que la colonne α^2 , et l'on supprime les cases débordées :

	α^2
σ_1	$\neg \text{Int}, 1$

Ajout des substitutions liées

On veut maintenant prévenir le cas où substituer α^k débordera toutes les cases d'une ligne, et transformera donc la ligne en la substitution identité, ce qui équivaut à créer le type vide. On cherche donc à ce que dans chaque ligne, après l'application de la substitution $\{\alpha^k \rightarrow \nu\}$, il existe au moins une case bornée.

Pour cela, nous allons uniquement nous intéresser aux lignes où α^k apparaît dans une borne et où toutes les autres cases sont débordées. On choisit arbitrairement une borne dans laquelle α^k apparaît, et :

- si c'est une borne inférieure i_j^i , on rajoute la colonne α^k de Tally $[(i_j^i, 0)]$ à notre tableau de contrainte.
- Si c'est une borne supérieure s_j^i , on rajoute la colonne α^k de Tally $[(1, s_j^i)]$ à notre tableau de contrainte.

Comme α^k est toujours le dernier élément de son ordre total, il n'y a pas de variable de type dans les substitutions.

Exemple

Pour $\alpha^1 \setminus \alpha^2$:

Tally $[(\alpha^1 \setminus \alpha^2, 0)] =$

	α^1	α^2
σ_1	$0, \alpha^2$	$0, 1$

On garde uniquement la colonne α^2 avec les cases bornées (ici aucunes):

α^2

Pour chaque ligne, on vérifie si toutes les cases sont débordées ou contiennent α^2 dans leur bornes (ici, la ligne σ_1) et on rajoute les tallying respectifs :

Tally $[(1, \alpha^2)] =$

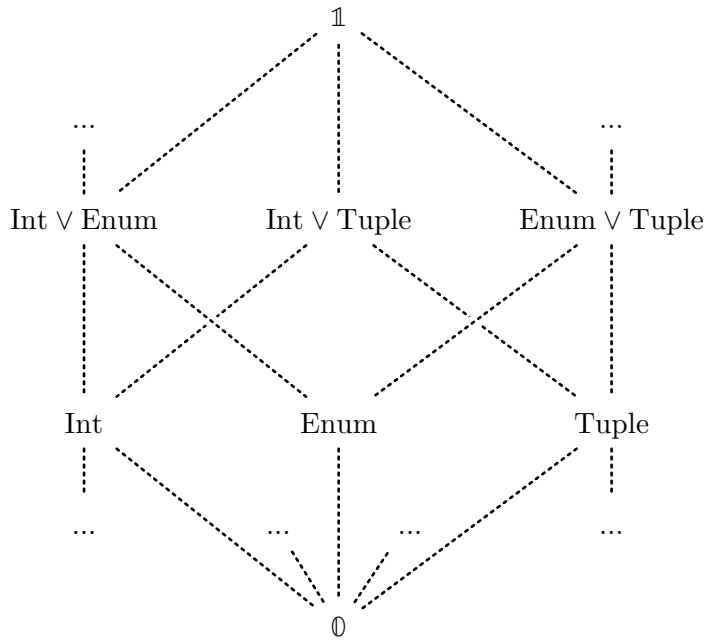
	α^2
σ_1	$0, 1$

On rajoute donc la colonne α^2 à notre tableau de contraintes (en rouge) :

α^2
$1, 1$

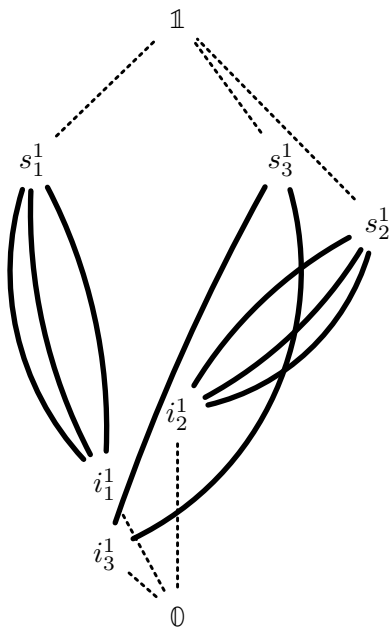
7.2.2 Créer ν respectant ces contraintes

On veut donc que pour toute case (i,s) du tableau de contrainte, $i \not\leq \nu \vee \nu \not\leq s$. Afin de mieux visualiser le problème, on peut voir notre système de type comme un treillis complet infini, et nos contraintes de types comme des chemins à « éviter » pour choisir un bon type :



Représentation des types ensemblistes sous forme de treillis. Tout les chemins ne sont pas représentés (car il y en a un nombre infini) et les chemins en pointillés indiquent qu'il peut y avoir d'autres types entre 2 types.

On peut donc représenter nos contraintes comme l'ensemble de tout les chemins entre une paire (i,s) :



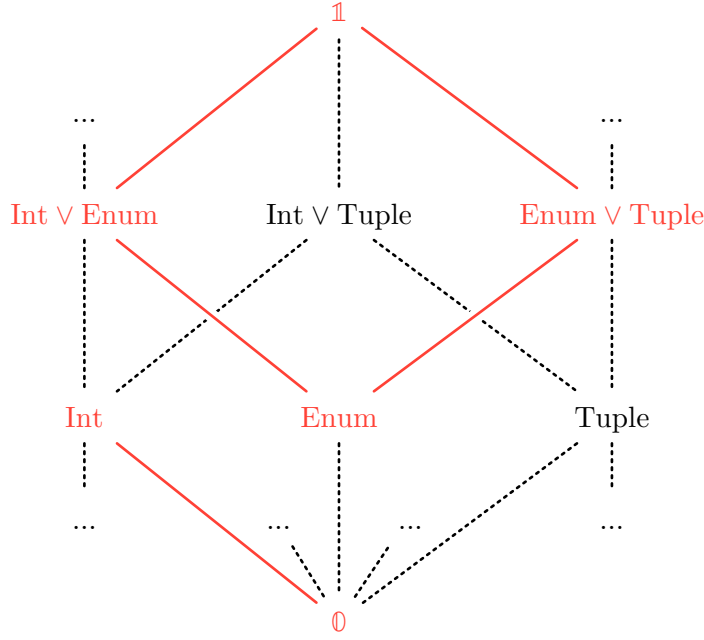
Créer un ν respectant ces contraintes revient donc à trouver un point n'étant sur aucun chemin de contrainte.

Exemple

On a le tableau de contrainte suivant :

α^1
$\emptyset, \text{Int}$
$\text{Enum}, \mathbb{1}$

Alors le treillis ressemblera à ça :



Donc des solutions comme "a", Tuple, $42 \vee (\text{Int}, \text{Enum})$ sont des bons ν , mais $\text{Enum} \vee [0, +\infty[$ ou $\mathbb{1}$ videraient le type.

Créer l'algorithme permettant de trouver ces types s'est avéré très compliqué, et cela n'a pas été fait dans le cadre de ce stage. Pour le reste du rapport, on considérera que l'on a un oracle `create_nu(ts)` qui prend en entrée un tableau `ts` correctement formé et renvoie en sortie ν tel $\forall (i, s) \in \text{ts}, i \not\leq \nu \vee \nu \not\leq s$.

Note

On considère un tableau de contrainte comme **correctement formé** si un tel ν existe, sinon cela voudrait dire qu'il n'existe aucun type ν tel que $t[\alpha^k \leftarrow \nu] \not\leq \emptyset$, ce qui est impossible.

Exemple

En continuant l'exemple de $\alpha^1 \setminus \alpha^2$, `create_nu([1, 1])` renvoie ν tel que $\mathbb{1} \not\leq \nu \vee \nu \not\leq \mathbb{1} \Leftrightarrow \nu \neq \mathbb{1}$. Donc n'importe quel type qui ne soit pas $\mathbb{1}$ est une solution acceptable.

7.3 Preuve de terminaison

Théorème 7.5

Pour tout type non-vidé, l'algorithme polyw termine en temps fini.

Démonstration: On peut distinguer 3 cas possibles pour notre algorithme :

- Si le type t n'a pas de variables de type : on renvoie la fonction identité, ce qui se fait en temps fini
- Si $\text{Tally}[(t, \emptyset)] = \emptyset$: On renvoie la substitution $\bigcup_{\alpha^i \in \text{Vars}(t)} \{\alpha^i \rightarrow \emptyset\}$, ce qui se fait aussi en temps fini
- Sinon :

Créer le tableau de contrainte se fait en temps fini, et la trouver le ν aussi. Comme il n'existait aucune variable de type dans les cases du tableau de contrainte, ν n'en contient pas non plus, donc $\text{Vars}(t[\alpha^k \leftarrow \nu])$ contient strictement moins d'éléments que $\text{Vars}(t)$. Donc par un ordre numérique sur la taille de $\text{Vars}(t)$, notre algorithme termine bien. $\square$

7.4 Preuves de sûretés

Théorème 7.6

Pour tout type t non-vidé, $\text{polyw}(t)$ renvoie une substitution σ tel que $\text{Vars}(t\sigma) = \emptyset \wedge t\sigma \not\leq \emptyset$.

Démonstration:

La terminaison de notre algorithme est fondée sur la diminution du nombre de variable de type, et nos 2 conditions d'arrêts s'assurent que $t\sigma$ n'ait plus de variable de type, il est donc trivial de montrer que $\text{polyw}(t) = \sigma$ renvoie bien une substitution telle que $\text{Vars}(t\sigma) = \emptyset$.

Montrons maintenant que l'algorithme ne peut pas renvoyer une substitution qui vide le type t :

Prouver que t ne deviens pas vide revient à prouver qu'aucune ligne du tableau de tallying ait toutes ses cases débordées, on va donc séparer cela en plusieurs cas :

Si la case α^k de la ligne est bornée : Comme le choix de ν s'assure que α^k sera substitué par quelque chose en dehors de ses bornes, la ligne est supprimée, donc on ne peut plus produire un type vide avec cette substitution.

Si la case α^k est débordée mais qu'aucune autre case de la ligne n'avait α^k dans ses substitutions : Il y avait forcément une autre case bornée dans la ligne, donc quelque soit le ν choisi, la substitution ne devient pas l'identité.

Si la case α^k est débordée, que α^k apparaît dans les bornes d'une autre case mais qu'une case dont les bornes ne contiennent pas α^k est bornée : Même si le choix de ν débordé d'autres cases, une case reste bornée, donc le type ne devient pas vide.

Si la case α^k est débordée, qu' α^k apparaît dans les bornes d'une autre case et que toutes les autres cases sont débordées : Grâce à la partie du tableau de contrainte où l'on ajoute les contraintes des substitutions liées, le choix de ν s'assure qu'au moins une case de la ligne ne soit pas débordée.

Donc quelque soit la forme de notre ligne de substitution, notre ν s'assure que la ligne soit supprimée où qu'elle ne soit pas entièrement débordée. Donc notre type ne peut pas devenir vide, donc pour tout type t non-vide, on a $\sigma := \text{polyw}(t)$ tel que $t\sigma \not\leq \emptyset$.

□

8 Un cas concret

Faisons un exemple complet avec $t = (\alpha^1, \neg\alpha^1) \setminus (\alpha^2, \alpha^3)$:

$\text{Tally}(t, \emptyset) =$

	α^1	α^2	α^3
σ_1	$\emptyset, \emptyset$	$\emptyset, \mathbb{1}$	$\emptyset, \mathbb{1}$
σ_2	$\mathbb{1}, \mathbb{1}$	$\emptyset, \mathbb{1}$	$\emptyset, \mathbb{1}$
σ_3	$\neg\alpha^3, \neg\alpha^3 \vee \alpha^2$	$\neg\alpha^3, \mathbb{1}$	$\emptyset, \mathbb{1}$

Donc notre tableau de contrainte ts est vide, car α^3 n'a que des cases débordées. α^3 apparaît dans σ_3 , et il n'y a pas de case bornée où α^3 n'apparaît pas, donc on fait $\text{Tally}(\neg\alpha^3, \emptyset) =$

	α^3
σ_1	$\mathbb{1}, \mathbb{1}$

Donc on l'ajoute à ts :

α^3
$\mathbb{1}, \mathbb{1}$

$\text{create_nu}(ts) = \neg 42$, donc on a la substitution $\{\alpha^3 \rightarrow \neg 42\}$.

$t[\alpha^3 \leftarrow \neg 42] = (\alpha^1, \neg\alpha^1) \setminus (\alpha^2, \neg 42)$, donc on recommence : $\text{Tally}(t[\alpha^3 \leftarrow \neg 42], \emptyset) =$

	α^1	α^2
σ_1	$\emptyset, \emptyset$	$\emptyset, \mathbb{1}$
σ_2	$\mathbb{1}, \mathbb{1}$	$\emptyset, \mathbb{1}$
σ_3	$42, 42 \vee \alpha^2$	$42, \mathbb{1}$

On crée le tableau de contrainte ts :

α^2
$42, \mathbb{1}$

Comme la case $\sigma_3(\alpha^2)$ est bornée, on a pas besoin de rajouter quoi que ce soit pour cette ligne.

$\text{create_nu}(ts) = 42$, donc on a la substitution $\{\alpha^2 \rightarrow \emptyset\}$.

$t[\alpha^3 \leftarrow \neg 42][\alpha^2 \leftarrow \emptyset] = (\alpha^1, \neg\alpha^1)$, donc on recommence : $\text{Tally}((\alpha^1, \neg\alpha^1), \emptyset) =$

	α^1
σ_1	$\emptyset, \emptyset$
σ_2	$\mathbb{1}, \mathbb{1}$

On crée le tableau de contrainte ts :

α^1
$\emptyset, \emptyset$
$\mathbb{1}, \mathbb{1}$

$\text{create_nu}(\text{ts}) = \neg 42$, donc on a la substitution $\{\alpha^1 \rightarrow \neg 42\}$. Donc on a $t[\alpha^3 \leftarrow \neg 42][\alpha^2 \leftarrow \emptyset][\alpha^1 \leftarrow \neg 42] = (\neg 42, 42)$, ce qui ne contient pas de variable de type, donc l'algorithme renvoie $\sigma = \{\alpha^1 \rightarrow \neg 42; \alpha^2 \rightarrow \emptyset; \alpha^3 \rightarrow \neg 42\}$ et $t\sigma = (\neg 42, 42)$.

On peut maintenant faire $\text{Witness}(\neg 42, 42) = (\text{Witness}(\neg 42), \text{Witness}(42)) = (41, 42)$.

Donc $t = (\alpha^1, \neg \alpha^1) \setminus (\alpha^2, \alpha^3)$ a pour habitant $(41, 42)$ avec la substitution $\{\alpha^1 \rightarrow \neg 42; \alpha^2 \rightarrow \emptyset; \alpha^3 \rightarrow \neg 42\}$.

9 Conclusion et perspectives

Au cours de ce stage, nous avons implémenté l'algorithme de witness pour les types monomorphes, prouvé sa sûreté, et avons grandement avancé sur l'algorithme permettant de retirer les variables de type d'un type polymorphe sans le rendre vide.

Une suite possible serait de trouver un algorithme implémentant $\text{create_nu}(\text{ts})$, afin de permettre une utilisation pratique de l'algorithme.

Bibliographie

- [1] G. Castagna, « Semantic subtyping: challenges, perspectives, and open problems », n° 3701, p. 1-20, 2005.
- [2] M. Laurent et K. Nguyen, « Implementing Set-Theoretic Types », nov. 2025, [En ligne]. Disponible sur: <https://hal.science/hal-05369012>
- [3] G. Castagna, K. Nguyen, Z. Xu, et P. Abate, « Polymorphic Functions with Set-Theoretic Types. Part 2: Local Type Inference and Type Reconstruction », nov. 2013, [En ligne]. Disponible sur: <https://hal.science/hal-00880744>
- [4] B. Courcelle, « Fundamental properties of infinite trees », *Theoretical Computer Science*, vol. 25, n° 2, p. 95-169, 1983, doi: [https://doi.org/10.1016/0304-3975\(83\)90059-2](https://doi.org/10.1016/0304-3975(83)90059-2).